Algorithms and Computational Complexity

Algorithm analysis: Time and Space Complexity, Mathematical introduction of asymptotic notations such as Big-O, Omega, Theta, recurrence relations, Applications of Master Theorem, Divide and conquer algorithms: Merge sort, quicksort, expected running time derivation of quicksort, sorting in linear time, radix sort, bucket sort, Fast integer multiplication: Karatsuba algorithm, Matrix multiplication: Naive method, Strassen's algorithm, analysis and applications, Heaps and priority queues: Binary heaps, heap operations, complexity, Hashing and amortized analysis: Hash functions, chaining, probing, potential method, Dynamic programming: LCS, matrix chain multiplication, edit distance, Bellman equations, Greedy strategies: Huffman coding, activity selection, MST algorithms (Kruskal, Prim), binary search, Graph traversal: BFS, DFS, topological sort, strongly connected components, Shortest paths and network flow: Dijkstra, Bellman-Ford, Graph algorithms in social networks: Community detection, centrality, PageRank, Complexity theory: Basic Complexity Classes, P, NP, NP-completeness, SAT, Knight's Tour, clique, vertex cover, TSP, The Turing Machine, The Turing's Halting Problem, Cryptographic Algorithms: Modular exponentiation, RSA, Diffie-Hellman, ECC overview, Randomized Computation, Probabilistic Turing Machines (PTM), Example of PTM: Probabilistic Primality Testing.

Reference Books:

1. Introduction to Algorithms (4th Ed.) by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, 2022.
2. Computational Thinking: First Algorithms, Then code by Paolo Ferragina and Fabrizio Luccio, 2018.
3. Algorithm Design by Jon Kleinberg and Éva Tardos, Pearson, 2005.
4. The Algorithm Design Manual by Steven S. Skiena, Springer, 2008.
5. Computational Complexity: A Modern Approach by Sanjeev Arora and Boaz Barak, Cambridge University Press, 2009.

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

Algorithms and Computational Complexity-LAB

Solving recurrence relations using symbolic computation in Python, Implementation and comparison of Merge Sort and Quick Sort using recursion and visualization, Karatsuba's algorithm and time it against native multiplication, Strassen's matrix multiplication: recursive implementation, Simulation of heap operations using Python libraries and custom implementation, Dynamic programming: LCS, matrix chain multiplication, Graph algorithms: BFS, DFS, topological sort, using adjacency structures, Implementation of Dijkstra, Bellman-Ford, Cryptographic and NP problems: RSA encryption/decryption, Probabilistic Primality Testing, Knight's Tour.

Reference Books:

1. Introduction to Algorithms (4th Ed.) by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, 2022.
2. Computational Thinking: First Algorithms, Then code by Paolo Ferragina and Fabrizio Luccio, 2018.
3. Algorithm Design by Jon Kleinberg and Éva Tardos, Pearson, 2005.
4. The Algorithm Design Manual by Steven S. Skiena, Springer, 2008.
5. Computational Complexity: A Modern Approach by Sanjeev Arora and Boaz Barak, Cambridge University Press, 2009.

Algorithm :

History :

Alg → Al. Khawarizmi

latinized form → Algorithmi de Numero Indorum

on the numbers of the Indians

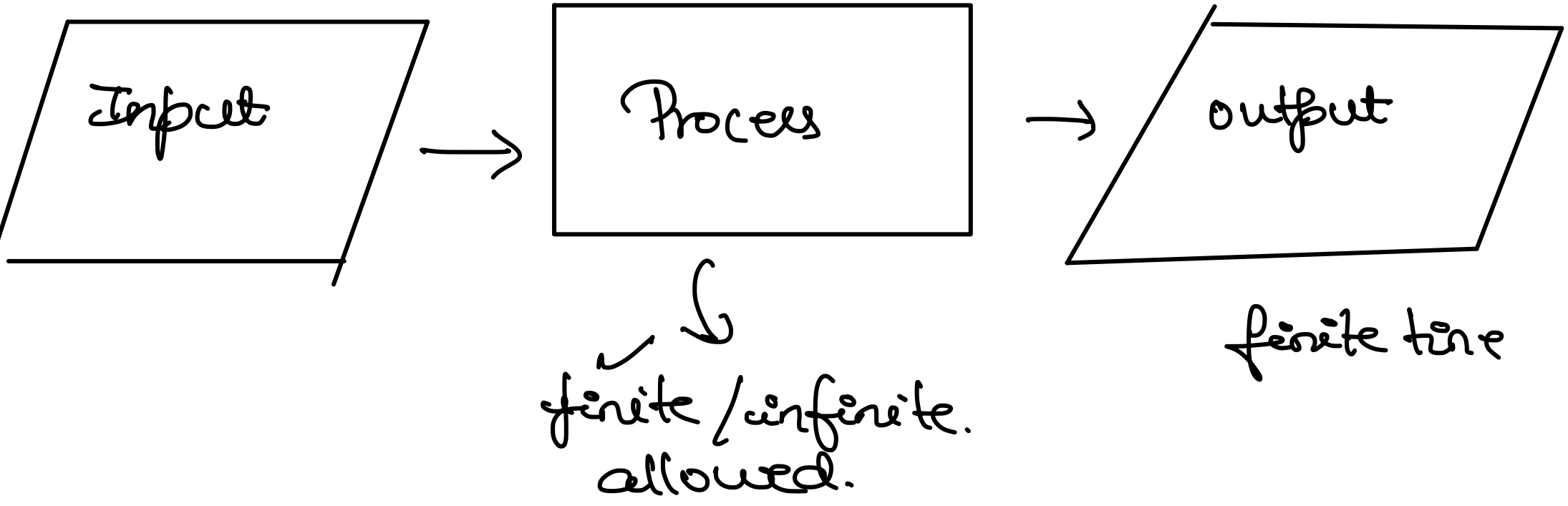
Algorithm:

Input:

Process: Well-defined instruction.

Output:

* Sequence of well-defined instructions with input and output



Def^{n}: Algorithm is a sequence of well-defined operations, taking an input, processing it to give an output in finite time.

Algorithm — Must be Correct.
— Finite no. of steps (Terminate).

n — size of the input.

$T(n) = ?$

Time Complexity is function of n (input size)

Analysis ← Time Complexity
Space Complexity.

(worst case).
↘ Not best case.

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

Mathematical volumes about algorithms rather than ordinary programming textbook.

↑ wrote

Donal Knuth (Art of Computer programming)

- Father of Analysis of Algorithms
- Creator of Tex
- Theoretical Computer Scientist and Mathematician.
- Turing Award Winner.

Q. posed to Barack Obama —

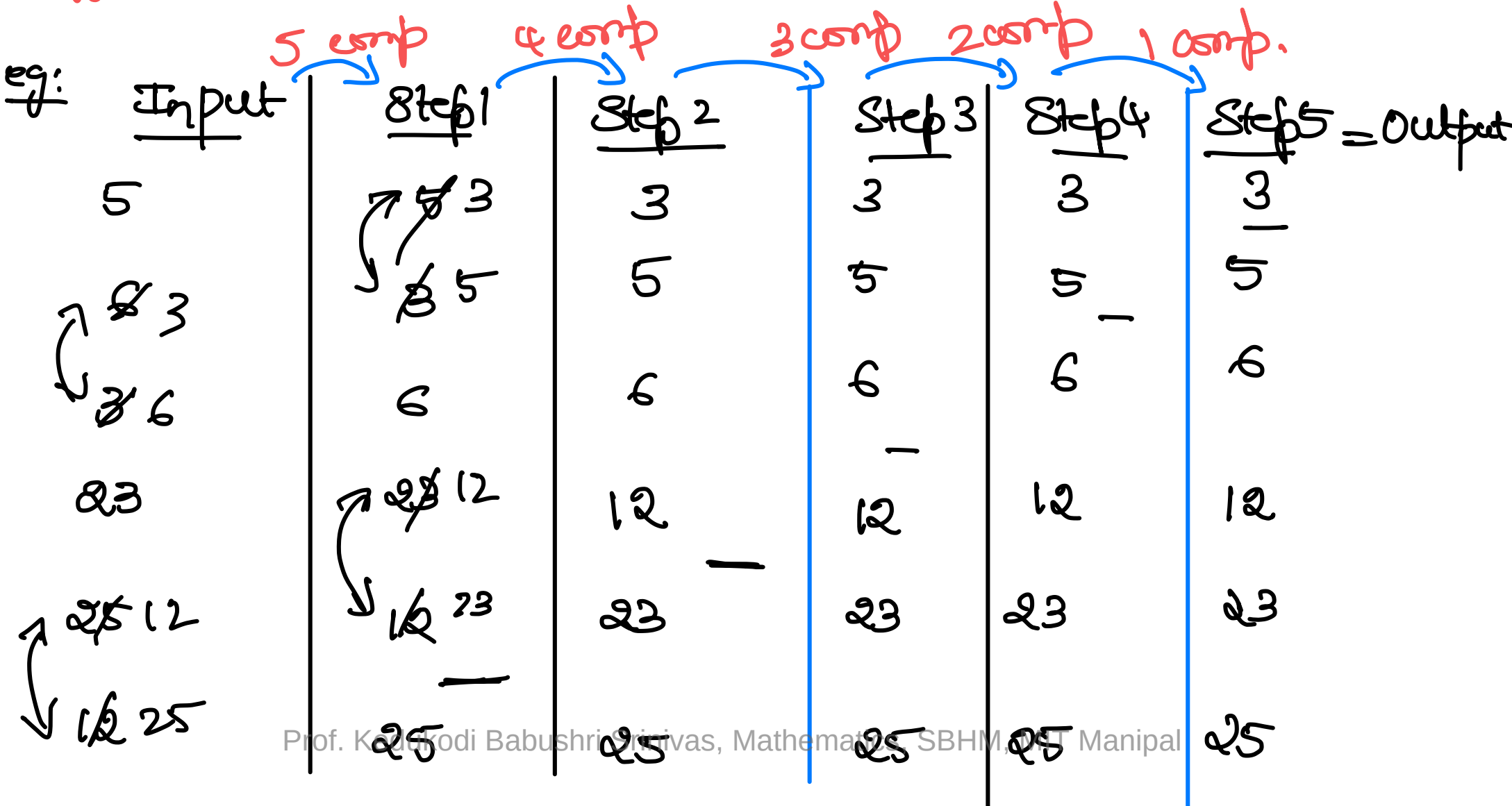
How to sort 10 million numbers in a good way?

I am sure Bubblesort is not a good way to go about it.

Bubblesort:

* Simplest one.

worst case: 5 exch. 4 exch. - - - -



Prof. Kodukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

$n=6$. Output will come in $n-1$ steps.

finite ✓

Analysis:

* Analysis of algorithms must be independent of hardware and software / programming language.

Q: What are the operations involved in eg?

i) Comparision (Basic operations)

ii) Exchanges

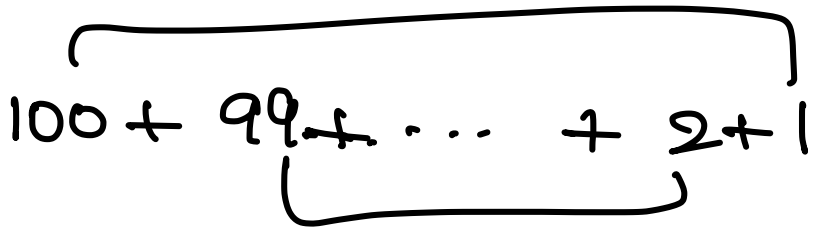
Q: How many comparisions?

Prof. Kedlaya Babushri Srinivas, Mathematics, SBHM, MIT Manipal

$5+4+3+2+1 = 15$

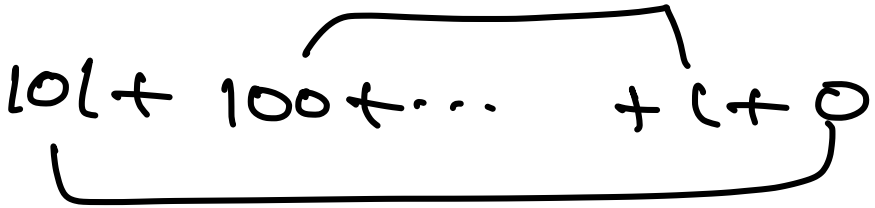
$\frac{n(n+1)}{2}$

Problem posed to Gauss when he was in school.



(50 pairs)

$101 \times 50 = 5050$



(51 pairs)

$101 \times 51 =$ 5151

repeats itself.

Total no of op (comparisions).

$(n-1) + \cdots + 1$ $\quad S = \frac{n(n-1)}{2}$

↑ last step.

$$S = 1 + 2 + \cdots + (n-1)$$

$$S = (n-1) + (n-2) + \cdots + 1$$

$$2S = (n-1)\,n$$

$$S = \frac{n(n-1)}{2} = T(n)$$

No of comparisions. given input n.

$\Rightarrow$ Time complexity.

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

(Workout an example)

Similarly, in the worstcase, the no of exchanges in step i is precisely $(n-i)$, $i=1,\ldots,n-1$.

$\therefore$ Total exchanges in worst case

$$= (n-1)+(n-2)+\cdots+1.$$

$$= \frac{n(n-1)}{2}.$$

Comparision of algorithms involves analysing the (Total.) found.

Time Comp

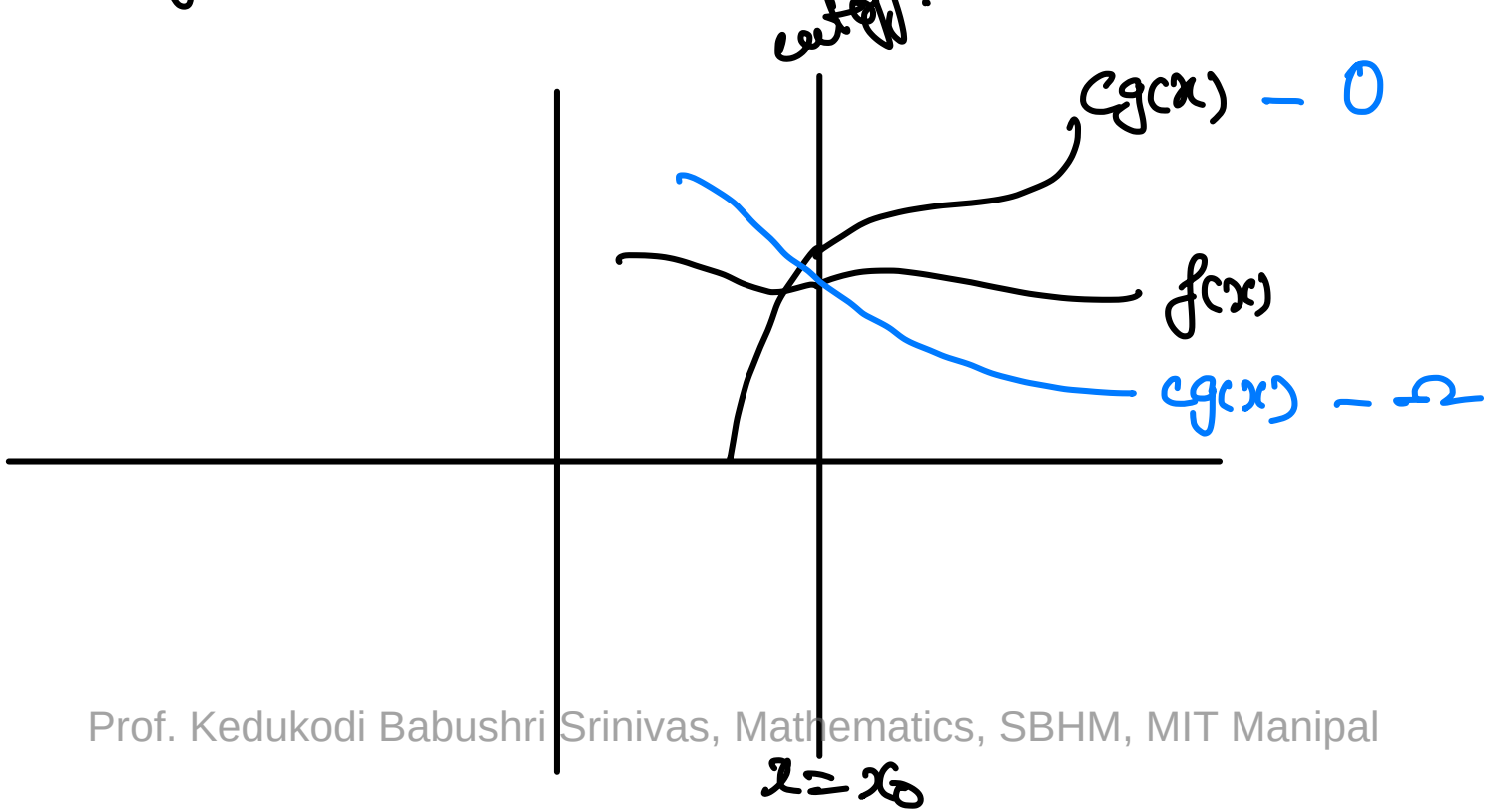
here, $\frac{n(n-1)}{2} \to$ poly. of degree 2

Prof. Kedukodi Babushri Srinivas, Mathematics, SBSHM, MIT Manipal

(Quadratic running time algorithm)

Bounded Above & Bounded below.

Want: $f(x) \leq c\,g(x)$ after some cutoff.



Defn 1:

($f(x)$ is bounded above by $g(x)$), $f(x) \in O(g(x))$ (Big O.)

if $\exists\ c>0$ and x_0 s.t $f(x) \leq c\,g(x)$

for all $x \geq x_0$.

Defn 2:

($f(x)$ is bounded below by $g(x)$), $f(x) \in \Omega(g(x))$ (omega)

if there exists $c>0$ and x_0 s.t $c\,g(x) \leq f(x)$

for all $x \geq x_0$.

Defⁿ 3 :

If $f(x) \in O(g(x))$ and $f(x) \in \Omega(g(x))$

then $f(x) \in \Theta(g(x))$

theta

i.e $f(x)$ and $g(x)$ have same time complexity.

Q: Prove that $\frac{n(n-1)}{2} \in \Theta(n^2)$

A: Let $f(n) = \frac{n(n-1)}{2}$, $g(n) = n^2$

T.S.T $f(n) \leq c\, g(n)$ for some $c > 0$

$$f(n) = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2} \leq \frac{n^2}{2} = cn^2 \quad \text{where}$$

$\leq$ atmost

$$= c \cdot g(n) \quad c = \frac{1}{2}$$

$\therefore f(n) \leq c\,g(n) \quad \forall \quad n \geq n_0 = 1.$ (i.e $\forall$ natural no's)

$\therefore$ true for all nat. no.

and $c = \frac{1}{2}$

$\therefore f(n) \in O(g(n))$

(Need not be unique)

Flexibility of choice is allowed for c & no.

H.W:

To prove: $f(n) \in \Omega(g(x))$

want:

$f(n) = \frac{n^2}{2} - \frac{n}{2} \geq \ldots \geq cn^2$ $\quad c = ?\ ;\ n_0 = ?$